

IA e Produtividade no Desenvolvimento

Do Modelo aos Agentes Autônomos de Código: Como o mercado evoluiu até o momento atual

Felipe Santana



INSTRUTOR & TECH LEAD

Liderança Técnica & Formação

Tech Lead na Netlex: Liderando equipes de alta performance na construção de produtos de software robustos baseados em processamento de documentos e IA.

Ex-Engenheiro no CESAR & XP Inc: Experiência prática em inovação corporativa, sistemas distribuídos e automação em grande escala.

Mentor na Residência do Porto Digital: Conectando a formação acadêmica das universidades com as demandas reais e práticas do ecossistema de tecnologia.

O que vamos discutir?

- Timeline da IA no mercado
- Como isso impacta os ciclos academicos
- Como os processos dentro de empresas foram completamente refeitos
- Como as empresas estão se adaptando
- Como novas empresas estão nascendo
- Quais métodos utilizar
- Para onde vamos

Trajectoria com IA (2018–2026)



Evolução do uso da IA

Evolução do uso ao longo de 12 meses

Autocomplete -> criação de testes -> leitura de código -> executor de prompts -> Engenharia de prompts -> planejador de tarefas -> agente autônomo

A IA no Dia a Dia e na Avaliação

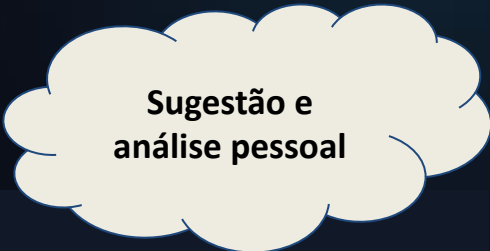
Na Startup (2025–2026)

Uso ostensivo de **Cursor IDE e Claude** no workflow diário de engenharia. Aplicação de provedores de IA para processamento inteligente de documentos jurídicos, OCR avançado, análise sintática e extração automatizada de dados operacionais.

Na Avaliação de Devs (2026)

Mudança de paradigma na **Residência do Porto Digital** e na liderança interna. O foco da avaliação de estagiários e liderados mudou da *velocidade de escrita manual de código* para a **capacidade de especificação, auditoria e senso crítico**.

Impacto em meios educacionais



Dimensão	Ensino Tradicional	Novo Ensino (Agentes & SDD)
Avaliação de Alunos	Decorar sintaxe e escrever código em papel/IDE	Avaliar capacidade de especificar, testar e auditar código
Uso de Ferramentas	Proibir IA por receio de plágio ou facilitação	Exigir uso de IA com defesa oral da arquitetura escolhida
Perfil do Estudante	Codificador básico focado em tarefas mecânicas	Engenheiro supervisor, autor de specs e auditor crítico
Papel do Professor	Corretor de erros de compilação e sintaxe	Mentor de arquitetura, lógica complexa e julgamento técnico

A Evolução Rápida das LLMs

Da Predição de Texto à Operação

Os modelos evoluíram rapidamente de geradores estatísticos de texto para sistemas com **capacidades operacionais reais**. Eles hoje raciocinam em múltiplos passos, executam chamadas de função (function calling) e manipulam ambientes de desenvolvimento.

A Evolução Rápida das LLMs

Nova Relação com a Engenharia

A IA deixou de ser apenas um autocomplete no editor ou uma busca no Stack Overflow. Ela se tornou um ****parceiro ativo de execução****, capaz de ler repositórios inteiros, entender dependências e agir no terminal sob supervisão humana.

Mudança no core de empresas e como elas nascem

Empresas AI Native

Muita gente executando ideias e criando empresas exclusivamente através da IA

Produtos subjacentes e de backoffice perdendo espaço

Produtos caros perdem relevância pois podem ser “clonados”

Empresas param de criar IA do zero

Não faz mais sentido criar algoritmos internamente dependendo do contexto

Empresas pivotando completamente

Muitas empresas com medo de ficar num nicho e não ter a mesma importância de uma empresa de IA

Como cada empresa está operando e se atualizando

- Software houses
- Empresas de produto
- Empresas de serviço
- Empresas de fora

Engenharia de Prompt para Devs

 **System Context & Roles:** Estabelecer claramente a stack técnica, padrões arquiteturais e premissas do projeto antes de pedir qualquer linha de código.

 **Escopo Reduzido e Focado:** Fornecer apenas as interfaces e arquivos relevantes no Context Window para evitar alucinações e perda de precisão.

 **Few-Shot Prompting Tecnológico:** Enviar exemplos reais de código no padrão do seu repositório (Input → Output desejado) antes da execução da tarefa.

 **Anti-Patterns e Restrições Explícitas:** Definir o que a IA *não* pode fazer (ex: "não altere a assinatura da função pública", "não use bibliotecas legadas").

Modelo, Harness e Agente



1. O Modelo (LLM)

O motor de raciocínio estatístico puro (Claude, GPT, Gemini). Recebe contexto e gera texto/código, mas não possui capacidade direta de interagir com o sistema sem auxílio externo.



2. O Harness (Estrutura)

A infraestrutura que envolve o modelo. Injeta o contexto do repositório, fornece ferramentas (linters, compilador, terminal) e gerencia a memória da sessão (Harness Engineering).



3. O Agente (Ciclo)

A união do Modelo com o Harness em um **loop autônomo**. O agente observa o resultado da ferramenta, decide o próximo passo, corrige erros e entrega a tarefa concluída.

Spec-Driven Development

Desenvolvimento Guiado por Specs

No SDD, o engenheiro não digita código nem faz prompts informais.

Ele constrói **arquivos de especificação estruturados (`\*.md`, `rules`)** contendo requisitos funcionais, contratos e cenários de teste.

O agente autônomo consome a especificação, implementa a solução no repositório e valida o resultado final contra a própria spec.

O foco da engenharia migra da sintaxe para a especificação precisa.

Managed Agents: Automating the Engineering

AUTONOMOUS EXECUTION



LEVEL 3: MANAGED AGENTS

Unlike self-hosted agents, Google manages all infrastructure, including sandbox provisioning and autonomous bug fixing.



THE EGRESS PROXY GUARANTEE

Protects private keys by intercepting outbound calls and injecting secrets only after verifying allowed domains.

INSIDE THE AGENTIC ENGINE



AUTONOMOUS SELF-CORRECT



As Novas Skills do Engenheiro



Visão Arquitetural

Entender a estrutura holística do software, dominar desacoplamento, padrões de API e modelagem de domínio — áreas onde o agente precisa do guia humano.



Auditoria & Code Review

O dev moderno atua como um parecerista sênior: avaliando segurança, eficiência de memória, débito técnico e consistência do código gerado pela IA.



Estratégia de Testes

Projetar cenários de teste complexos e edge cases que garantam que a implementação do agente satisfaça rigorosamente as regras de negócio.

Loop Engineering e Autovalidação

O Ciclo do Loop Engineering

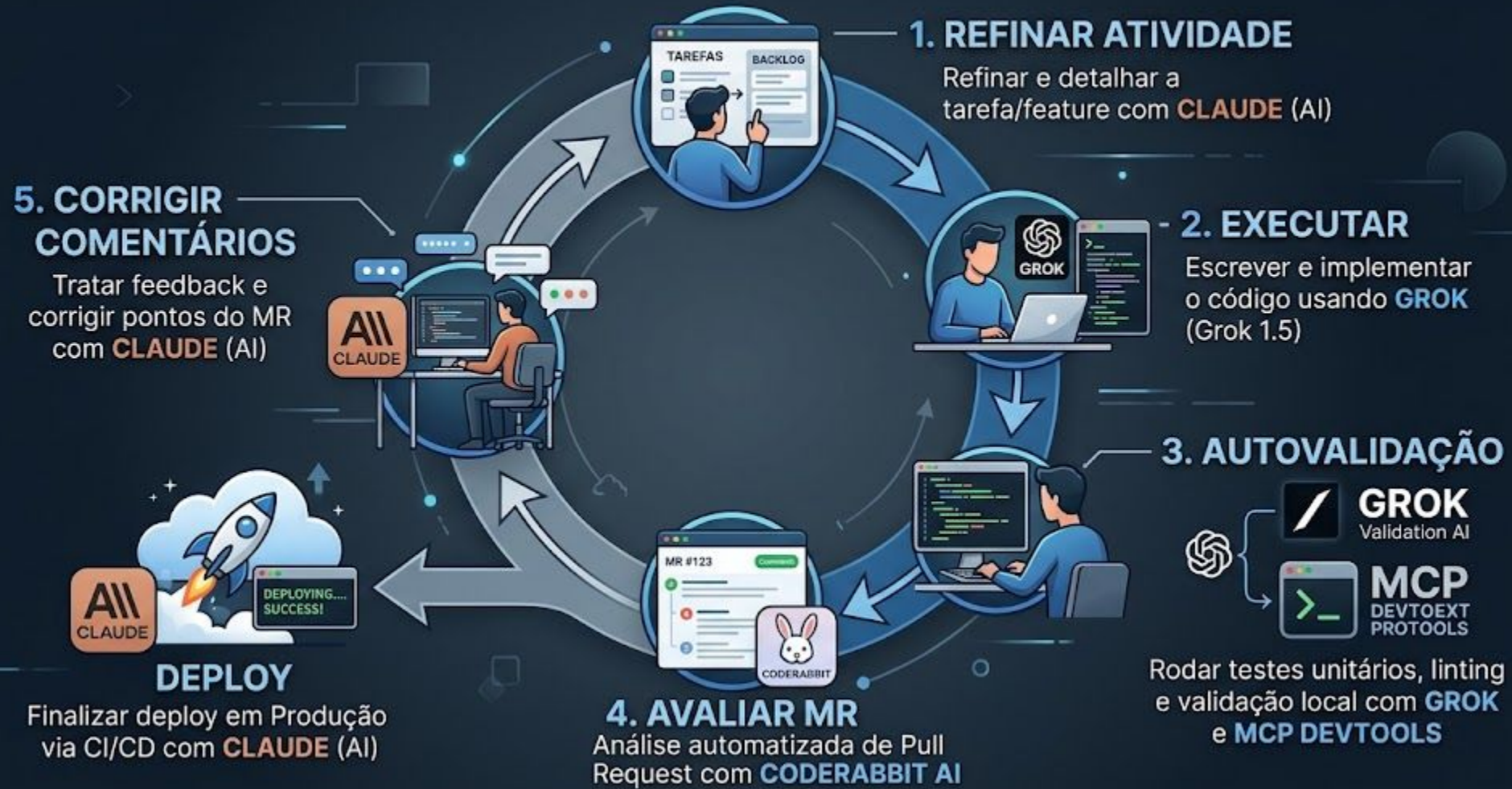
O desenvolvimento não é mais uma interação única (Prompt → Resposta). É um **loop contínuo** gerenciado por ferramentas automatizadas:

- ▶ 1. O dev define o objetivo e as especificações.
- ▶ 2. O agente altera o código e cria os arquivos.
- ▶ 3. O Harness executa o linter e os testes unitários.
- ▶ 4. O log de erro é realimentado ao modelo automaticamente.

Autovalidação na Prática

Ferramentas como Cursor e Claude Code utilizam esse loop interno para que o agente **identifique seus próprios erros de compilação e tipagem** antes mesmo de devolver a tarefa para a revisão final do engenheiro humano.

CICLO DE DESENVOLVIMENTO: FLUXO DE TRABALHO E INTEGRAÇÃO DE IA



Casos de uso de skills

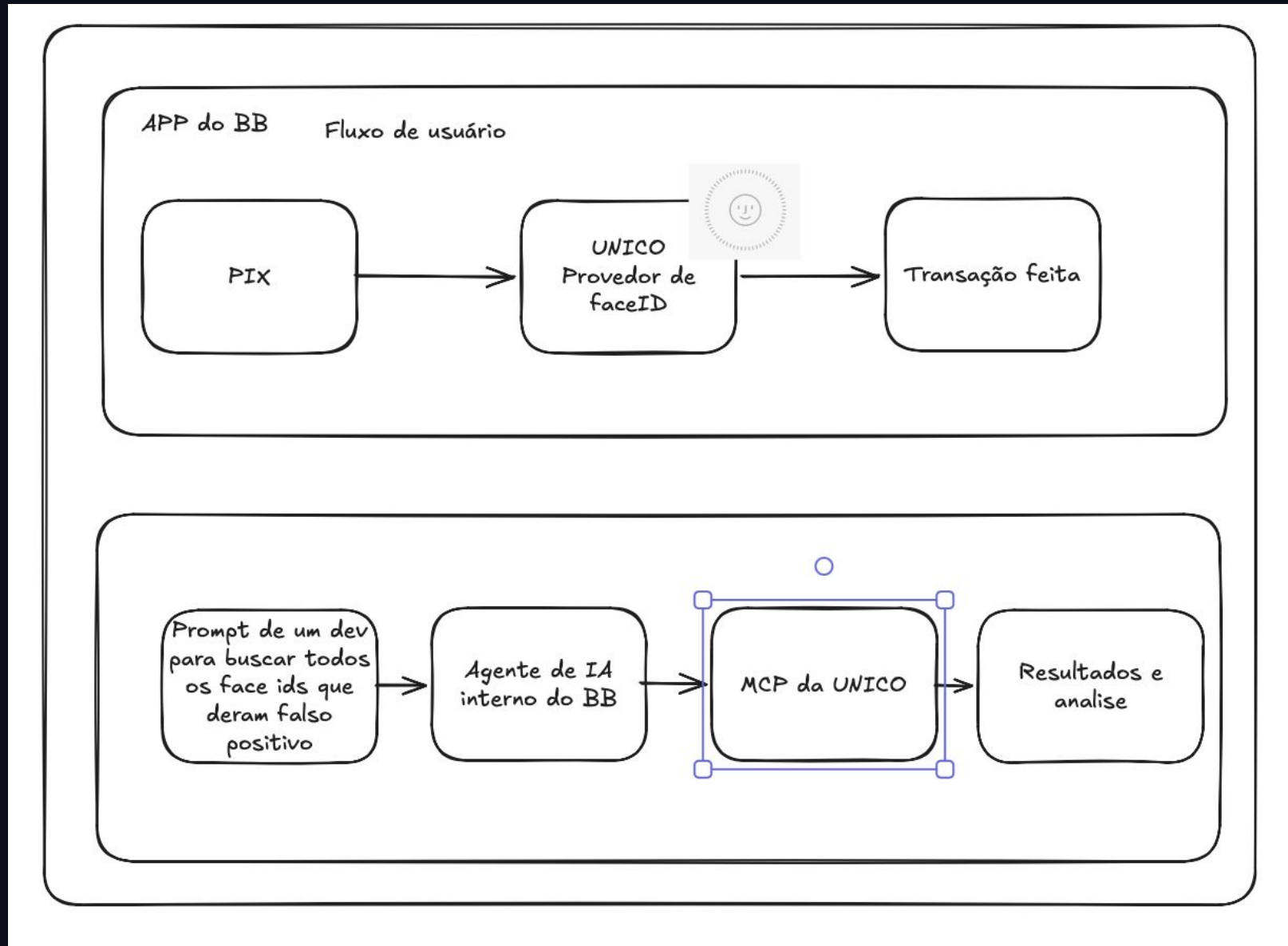
1. Automação de fluxos de trabalho repetitivos (ex: preparar um commit/PR)

Em vez de explicar toda vez o mesmo processo, você cria uma skill (tipo `/ship`) que roda os testes, o linter, olha o git diff e já rascunha uma mensagem de commit no padrão conventional-commit. Uma vez criada, ela fica disponível no comando `/ship`.

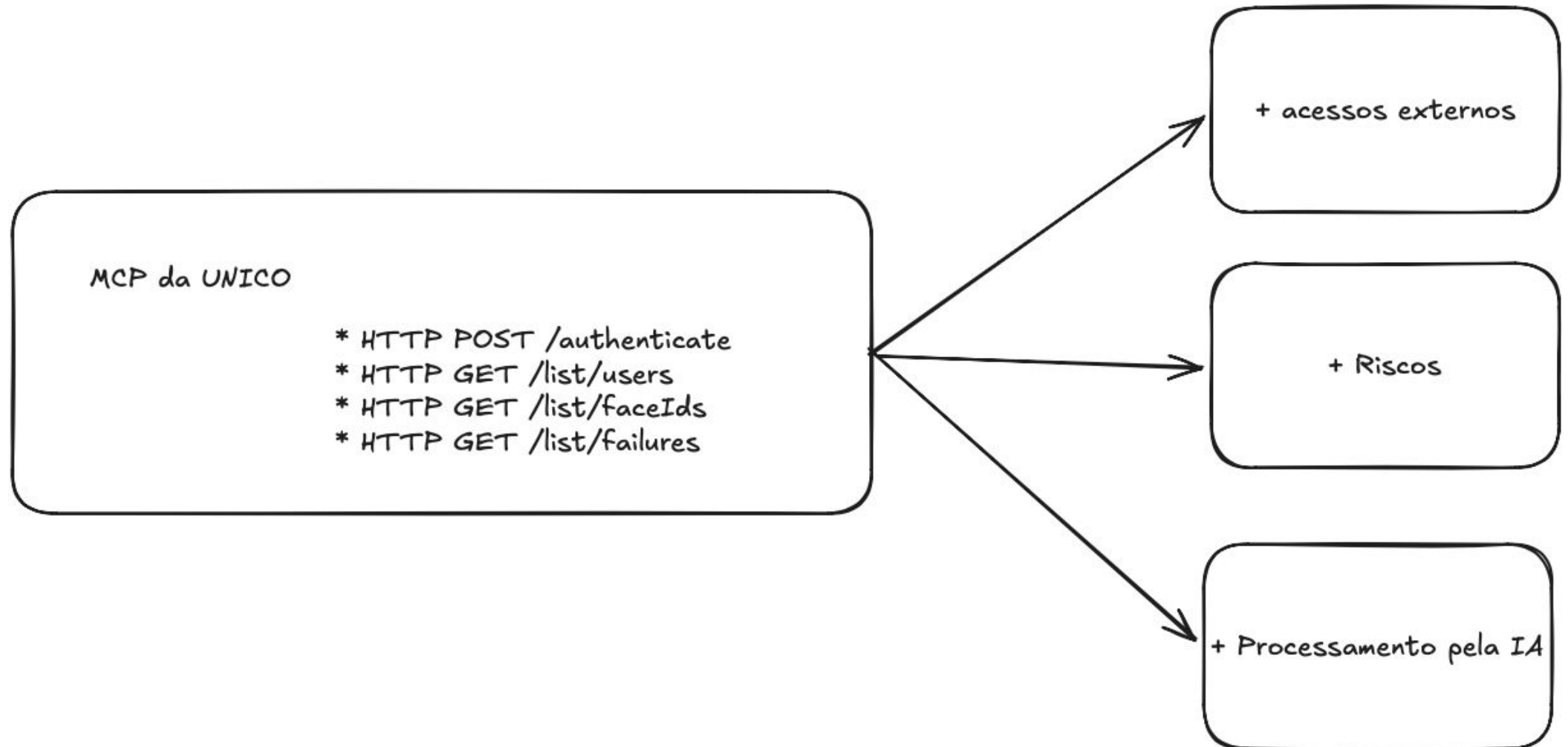
2. Definição de feature e modo de desenvolvimento

Selecionar skills que condizem com aquela feature e como o desenvolvedor quer implementá-la

Caso de usos de MCP – Acesso a dados de face Id de um provedor



Caso de usos de MCP – Acesso a dados de face Id de um provedor



Burocracias, dados e responsabilidades da IA

Dados anonimizados

IA só acessa dados anônimos e não identificáveis

Agente com mascaras e boundaries

Agente pre-definidos e pre-configurados para uso interno e já disponível com restrições

Quem é o dono do código fonte?

Anthropic, Google, Microsoft deixam explícito nos termos de uso que o usuário é proprietário de todo código fonte gerado

Perguntas & Debate

?